

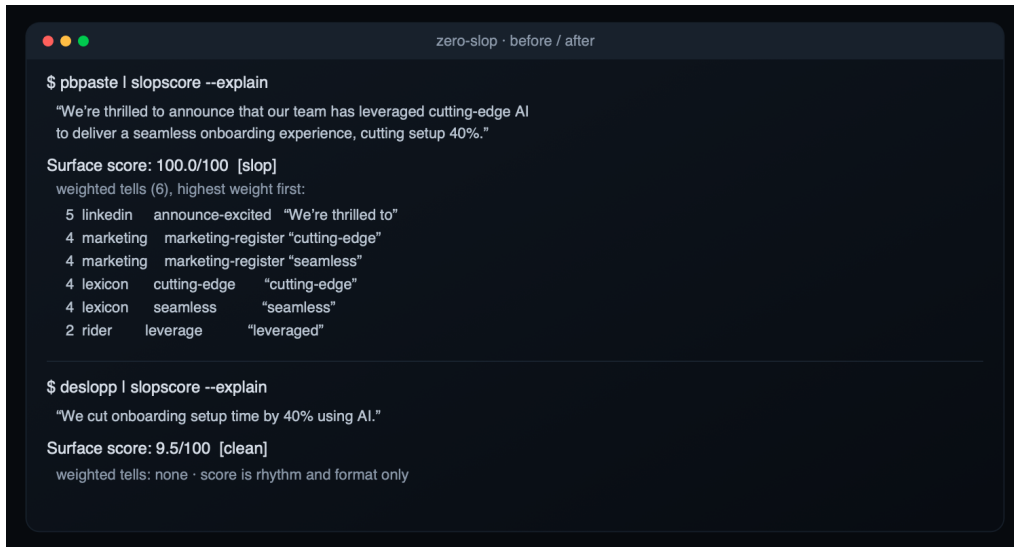
Zero Slop

You used AI to help with your writing, and now it reads like a machine wrote every word. You can hear it, and so can everyone who reads it. That machine sound has a name: slop.

Zero Slop finds that slop in your draft and takes it out without changing what you said. Its transparent surface score runs from 0 to 100 and shows which phrases and document-level signals contributed to it. After the rewrite, a copy editor fixes the mechanics. A second editor reads the result aloud and fixes its flow; the final checks make sure the facts survived.

```
npx skills add manavmishra/ZeroSlop --global
```

Then open your agent and say "de-slop this." The Python scorer needs no account or server and does not transmit your writing.



```
zero-slop · before / after

$ pbpaste | slopscore --explain
"We're thrilled to announce that our team has leveraged cutting-edge AI
to deliver a seamless onboarding experience, cutting setup 40%."

Surface score: 100.0/100 [slop]
weighted tells (6), highest weight first:
  5 linkedin  announce-excited  "We're thrilled to"
  4 marketing  marketing-register  "cutting-edge"
  4 marketing  marketing-register  "seamless"
  4 lexicon    cutting-edge      "cutting-edge"
  4 lexicon    seamless         "seamless"
  2 rider     leverage         "leveraged"

$ deslopp | slopscore --explain
"We cut onboarding setup time by 40% using AI."

Surface score: 9.5/100 [clean]
weighted tells: none - score is rhythm and format only
```

Why it matters now

Sounding like AI can cost credibility, and platforms are adding ways to report, classify, or limit repetitive machine-made material. LinkedIn added an AI-slop report signal; Snap and YouTube apply recommendation or monetization rules to some synthetic or mass-produced content; Substack lets readers scan text for AI signals. Those are different policies, not proof of one universal reach penalty. The practical point is simpler: readers notice generic machine prose, and writers deserve a way to inspect and fix it before publishing.

Who it is for

Anyone who writes under their own name and would rather not sound like a robot. A founder writing a launch post, a marketer shipping five things a week, a researcher who needs their paper to stay formal, an engineering team that wants slop to fail a check the way a bug does.

How it decides

First, it measures. Four surface channels examine known phrases, sentence rhythm, followability, and a combined formatting-and-register signal. The result is a heuristic surface score, not the probability that AI wrote the text. Most signals describe structure, so a synonym swap cannot resolve the structural issues. One em dash carries little weight; clusters of independent signals matter more. A separate predictability probe asks your assistant to guess masked words and reports how often the original word appears among its top three guesses. Zero Slop ships no model of its own. For important work, it drafts two or three approaches, rejects any that add or lose a fact, and

sends the cleanest version through the copy desk and read-aloud pass.

What makes it different

It is built on four open-source projects that worked out this craft first: no-ai-slop, humanizer, de-slop, and stop-slop. It adds three things a plain rewrite lacks.

A score you can inspect. The rules and structural measures are visible, and the same threshold check can run in a build. It is a repeatable editorial meter, not authorship proof.

A promise about your facts. A rewrite can invent a detail or drop a number to make a sentence flow. Zero Slop lists every figure, name, quote, and link in your original and rejects a version that loses one or adds one. A missing number may be obvious; an invented one can read naturally enough to go unnoticed, which is why the check exists.

A tool that learns from later edits. The editorial loop handles the current draft. A separate private learning loop can compare the delivered version with a later edit: a phrase you cut may be a tell it missed, while flagged text you keep may be a false alarm. No detection rule or preferred fix becomes active after a single edit pair. A potential phrase rule needs the same cut in three content-distinct edit pairs, followed by a novelty check and a safety check against reference human writing; single words need five edit pairs. Repeated replacements can also become private rewrite guidance after the same replacement recurs in three content-distinct edit pairs. The private learning file loads on the next run. Later evidence reconfirms detection rules and preferred fixes; without reconfirmation, stale detection rules decay and stale preferred fixes retire.

This is post-deployment, human-in-the-loop online learning. It adapts detection and fixing through inspectable local rules and preferences. Human corrections provide feedback, but Zero Slop does not perform reinforcement learning or RLHF, nor does it retrain the host model. Shared changes still require review, tests, versioning, and a release. Each session can check for a newer release with a metadata-only version query. That query never sends the draft.

What we tested

We wrote 50 AI-heavy drafts and compared four rewrites of each. No human raters took part. The records identify the evaluators only as LLM runs from one model family. Each of the two passes used five separate runs. The method names were hidden; the model saw the brief, source draft, factual inventory, and four rewrites, then rated human-likeness, voice, fidelity, craft, and platform fit.

The results moved between passes. The model selected Zero Slop for 32 of 50 drafts in the first and 23 in the second; it selected blader/humanizer for 18 and 22. The pooled counts were 55 and 40, but the passes agreed on a winner for only 26 drafts, and the head-to-head difference was not statistically significant. The public harness also omits the exact model, settings, and full prompt. The results show that the two systems were competitive on our synthetic set, not that either is generally better. We also wrote the scorer's discrimination set, so it serves as a regression check rather than a real-world accuracy estimate. A 1,000-document speed check runs in CI and must finish within 60 seconds.